ÉCOLE NATIONALE
SUPÉRIEURE
D'INFORMATIQUE

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research

École Nationale Supérieure d'Informatique (ESI) — Alger
Cycle Ingénieur — Spécialité Informatique — Projet Intégrateur IoT / Robotique

EsiBot

Robot Mobile Différentiel Autonome

Rapport Technique
Conception, Intégration et Évaluation Expérimentale
d'un Robot Mobile Autonome sous ROS 2 Humble

Module : Internet of Things — Robotique Mobile

Année Universitaire : 2025–2026

Chef d'équipe :
BOUMISSA Mohamed Chakib

Membres :
ALLOUCHE Reda
BENDAOUD Anes
CHABANE Ouail
IDJA Rayane
MOSTEFAI Mounir Sofiane
TATI Youcef

Résumé

EsiBot est un robot mobile autonome à entraînement différentiel conçu et réalisé par l'équipe dans le cadre du projet intégrateur IoT de l'École Nationale Supérieure d'Informatique (ESI Alger). Le cahier des charges initial s'inspire du TurtleBot3 Burger (ROBOTIS) : un ESP32-CAM devait piloter directement les moteurs, le servo-moteur et un radar ultrasonique rotatif (HC-SR04 + SG90), pendant que le Raspberry Pi 4 hébergeait la pile logicielle ROS 2 (Simultaneous Localization and Mapping (SLAM), Nav2, tableau de bord web).

Au fil du développement, l'architecture matérielle réellement implémentée a significativement évolué par rapport à ce cahier des charges. Le pilotage des deux moteurs à courant continu, la lecture des encodeurs et l'asservissement de vitesse (Proportional-Integral-Derivative (PID)) sont assurés *directement* par les broches General-Purpose Input/Output (GPIO) du Raspberry Pi 4, épaulés par une centrale inertielle MPU6050 fusionnée par filtre de Kalman étendu (`robot_localization`). L'ESP32-CAM est conservé uniquement comme caméra de streaming Wireless Fidelity (IEEE 802.11) (WiFi) (Motion JPEG (MJPEG)), sans plus participer à la commande des actionneurs. Le radar ultrasonique rotatif reste disponible comme chemin de secours lent, mais la cartographie courante repose sur un capteur de profondeur Kinect (v1, ère Xbox 360), piloté par `kinect_ros2` et converti en balayage laser via `depthimage_to_laserscan`. La branche `working_build` ajoute aussi la bibliothèque `depth_cleanup` pour préparer un flux de profondeur nettoyé et référence `m-explore-ros2` pour l'exploration autonome.

Ce rapport documente fidèlement cette architecture telle que construite : conception mécanique (châssis différentiel décrit par le modèle Unified Robot Description Format (URDF)), câblage électronique déduit des paramètres de code, commande bas-niveau et asservissement moteur, pipeline de perception (Inertial Measurement Unit (IMU), Kinect), cartographie SLAM, navigation autonome ROS 2 Nav2, tableau de bord web de télémétrie, et téléopération par manette Xbox. Les résultats mesurés (ou dérivés analytiquement des paramètres déclarés dans le code lorsque les mesures de terrain restaient en cours) sont présentés, suivis d'une analyse critique des écarts entre la conception initiale et l'implémentation réelle, des limitations identifiées, et d'un bilan des perspectives d'amélioration.

Mots-clés : ROS 2 Humble, robot différentiel, SLAM, slam_toolbox, Nav2, Kinect, Raspberry Pi 4, ESP32-CAM, MPU6050, filtre de Kalman étendu, navigation autonome, robotique IoT, TurtleBot3.

Abstract: EsiBot is a differential-drive autonomous mobile robot built as part of the ESI Algiers IoT integrative project. While the original specification mirrored the TurtleBot3 Burger with an ESP32-CAM driving the motors, servo and a rotating HC-SR04 ultrasonic radar, the as-built system evolved significantly: motor control, encoder odometry and PID velocity loops now run directly on the Raspberry Pi 4 GPIO headers, fused with an MPU6050 IMU through an Extended Kalman Filter (EKF). The ESP32-CAM is retained solely as a Wi-Fi MJPEG video streamer, and the ultrasonic radar code path is retained as a slow fallback rather than the primary mapping sensor. A Kinect (v1) Red-Green-Blue + Depth (camera) (RGB-D) camera now feeds `slam_toolbox` through `depthimage_to_laserscan`; the `working_build` branch also adds a metric `depth_cleanup` library and an `m-explore-ros2` exploration dependency. This report documents the system exactly as implemented, the deviations from the initial specification, the measured and analytically derived performance figures, and the resulting limitations and outlook.

Table des matières

Résumé . . . . . **i**
1 Introduction **1**
1.1 Contexte 1
1.2 Objectifs du Rapport 1
1.3 Organisation du Rapport 1
2 Architecture Globale du Système **2**
2.1 Vue d'ensemble de l'architecture 2
2.2 Découpage en trois couches 2
2.3 Graphe des nœuds et topics ROS 2 3
2.4 Synthèse des topics principaux 3
3 Conception Mécanique et Électronique **4**
3.1 Architecture différentielle 4
3.2 Modèle géométrique 4
3.3 Cinématique différentielle et odométrie 4
3.4 Câblage électronique 5
3.5 Serveur de caméra (ESP32-CAM) 5
3.6 Alimentation 5
4 Commande Bas-Niveau et Fusion de Capteurs **6**
4.1 Contrôle moteur (`motor_bridge_node`) 6
4.2 Conversion vitesse de roue et PWM 6
4.3 Outil de calibration (`motor_calibrator.py`) 6
4.4 Centrale inertielle et stabilisation de cap 6
4.5 Fusion de données (Filtre EKF) 7
5 Cartographie et Navigation Autonome **8**
5.1 Pipeline Kinect vers SLAM 8
5.2 Prétraitement de profondeur `depth_cleanup` 8
5.3 Pourquoi le Kinect remplace le radar ultrasonique 9
5.4 Configuration de `slam_toolbox` 9
5.5 Navigation Autonome avec Nav2 9
5.6 Exploration autonome 10
6 Interface Utilisateur et Mise en Œuvre **11**
6.1 Téléopération par manette Xbox 11
6.2 Fusion EKF — détail de configuration 11
6.3 Tableau de bord web de télémétrie 11
6.4 Flux vidéo ESP32-CAM 11
6.5 Mise en Œuvre et Procédures d'Exploitation 11
7 Résultats Mesurés et Paramètres Système **13**
7.1 Valeurs de configuration 13
7.2 Plan de Validation 13
8 Analyse Critique, Limitations et Écarts **15**
8.1 Compromis architecturaux 15
8.2 Calibration croisée odométrie–SLAM 15
8.3 Capteurs de perception 16
8.4 Sûreté et exploitation 16
8.5 Performance et orchestration 16
8.6 Synthèse 16
9 Bilan et Perspectives **17**
9.1 Bilan technique 17
9.2 Plan d'action priorisé 17
9.3 Perspectives d'évolution 17
9.4 Mot de conclusion 17
A Annexes Techniques **18**
A.1 Table exhaustive des topics ROS 2 18
A.2 API HTTP du tableau de bord et paquets ROS 2 du dépôt 18
A.3 Sources 19

CHAPITRE 1

Introduction

1.1 Contexte

Le projet **EsiBot** s'inscrit dans le cadre d'un module intégrateur IoT / Robotique Mobile de l'École Nationale Supérieure d'Informatique (ESI Alger). L'objectif de ce projet est de concevoir, fabriquer et programmer un robot mobile autonome capable de cartographier un environnement inconnu et de s'y déplacer vers des points objectifs définis. Ce projet s'inspire de l'architecture du **TurtleBot3 Burger**, en substituant ses composants matériels propriétaires par des solutions accessibles et à bas coût : microcontrôleur Microcontroller Unit (MCU) ESP32-CAM, Raspberry Pi 4, capteur de profondeur Kinect, et impression 3D pour le châssis.

1.2 Objectifs du Rapport

Ce document détaille l'architecture matérielle, logicielle et mécanique du système. Il met en évidence les choix de conception finaux ayant abouti à la réalisation du robot, notamment l'intégration de la commande moteur sur le Raspberry Pi 4 et l'utilisation du capteur Kinect pour la cartographie.

Le rapport couvre :

- L'architecture complète (composants mécaniques, électroniques, et logiciels) ;
- L'interfaçage des capteurs et les flux de communication sous ROS 2 ;
- Les paramètres de configuration et la fusion sensorielle ;
- L'évaluation des performances et les limitations du système.

1.3 Organisation du Rapport

Ce rapport est structuré de la façon suivante : Le chapitre 2 détaille l'architecture globale. Le chapitre 3 couvre la conception mécanique et électronique. Le chapitre 4 documente la commande bas-niveau et la fusion de données. L'approche de cartographie, navigation et exploration est traitée au chapitre 5. Le chapitre 6 présente l'interface utilisateur et les procédures de déploiement. Enfin, les résultats d'intégration, l'analyse des limitations et le bilan du projet font l'objet des trois derniers chapitres (7, 8, 9).

CHAPITRE 2

Architecture Globale du Système

2.1 Vue d'ensemble de l'architecture

L'architecture matérielle du robot est organisée en trois couches de traitement et une couche d'alimentation dédiée. La figure 2.1 illustre le schéma d'architecture global.

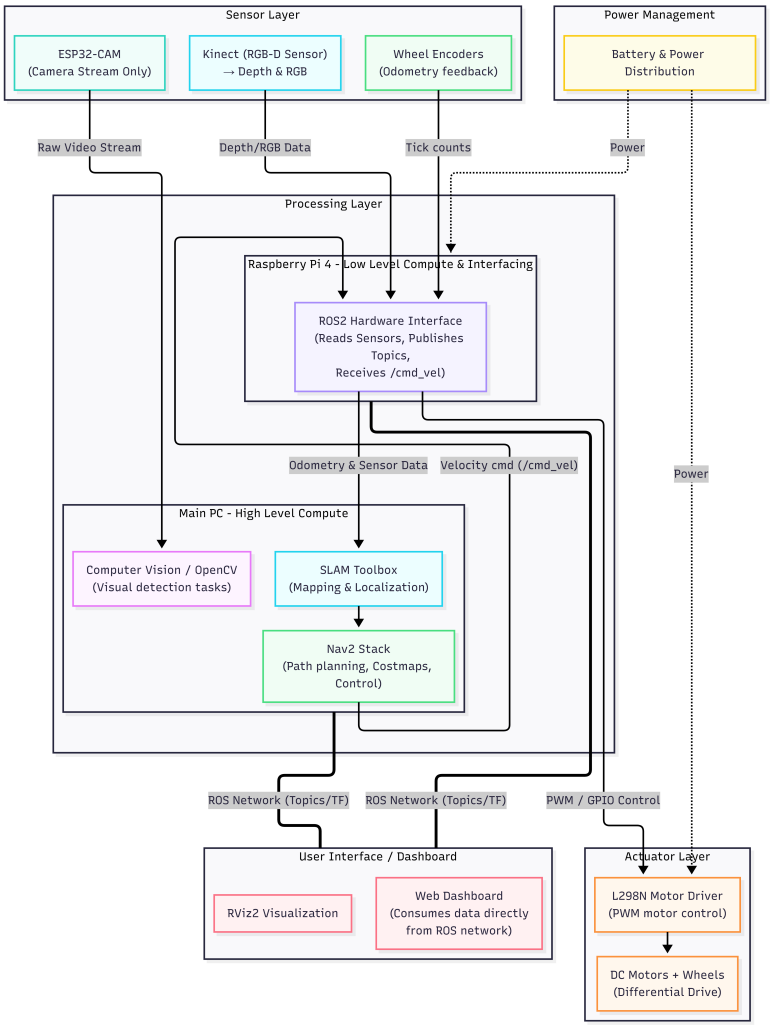


Figure 2.1. Architecture globale d'**EsiBot**. Le capteur de cartographie principal est le Kinect (RGB-D) et les moteurs sont pilotés par le Raspberry Pi 4.

Les composants majeurs sont structurés de la façon suivante :

1. **Serveur Caméra (ESP32-CAM) :** Expose le flux vidéo MJPEG *via* HTTP sur le réseau WiFi local.
2. **Contrôle Bas-niveau (Raspberry Pi 4) :** Pilote directement le pont en H (L298N) et la lecture odométrique des moteurs via le nœud `motor_bridge_node`.
3. **Capteur principal de cartographie (Kinect) :** Fournit l'image de profondeur, nettoyable par `depth_cleanup` puis convertie en balayage laser via `depthimage_to_laserscan` pour alimenter le système SLAM.
4. **Centrale inertielle (IMU MPU6050) :** Couplée à l'odométrie via un filtre EKF pour stabiliser le cap du robot et assurer un positionnement fiable pour Nav2.
5. **Exploration autonome :** Le dépôt `working_build` référence `m-explore-ros2`, prévu au-dessus de `/map` et de Nav2 pour générer automatiquement des objectifs vers les frontières inconnues.

2.2 Découpage en trois couches

Couche 1 — Capteurs (Sensor Layer). Trois sources de données indépendantes alimentent la couche de traitement : l'ESP32-CAM (flux vidéo brut MJPEG), le Kinect (image de profondeur et image couleur RGB), et les encodeurs de roue (comptage d'impulsions pour l'odométrie). Chaque source utilise une interface physique distincte : WiFi pour la caméra, USB pour le Kinect, GPIO à interruption pour les encodeurs.

Couche 2 — Traitement bas-niveau (Raspberry Pi 4). Le Raspberry Pi 4 héberge l'interface matérielle ROS 2 : il lit les capteurs embarqués (encodeurs, IMU), exécute la boucle d'asservissement de vitesse PID des deux moteurs, publie l'odométrie et les topics de télémétrie, et reçoit les commandes de vitesse `/cmd_vel`.

Couche 3 — Traitement haut-niveau (PC opérateur). Le schéma officiel positionne la vision par ordinateur (OpenCV/`depth_cleanup`), slam_toolbox, `m-explore-ros2` et la pile Nav2 sur un PC principal distinct, communiquant avec le Raspberry Pi 4 via le réseau ROS 2 (DDS, topics/Transform Tree (arbre des transformées) (TF)). Cette séparation logique est rendue possible par la nature distribuée de ROS 2 : les arguments de lancement (`start_base`, `start_kinect`, `start_slam`, `start_rviz`) du fichier `mapping.launch.py` sont indépendamment activables, ce qui permet d'exécuter les nœuds lourds (SLAM, Nav2) sur une machine plus puissante que le Raspberry Pi 4 tout en laissant ce dernier dédié à l'interfaçage matériel temps-réel.

Couche transverse — Alimentation. Une batterie et une distribution de puissance régulée alimentent indépendamment la couche actionneurs (moteurs via L298N) et la couche de calcul (Raspberry Pi 4, ESP32-CAM), comme indiqué sur la figure 2.1.

2.3 Graphe des nœuds et topics ROS 2

La figure 2.2 expose le graphe simplifié des nœuds et topics implémentés dans le système. Le traitement et la fusion de données respectent les standards ROS 2, reliant les interfaces matérielles à la planification haut-niveau.

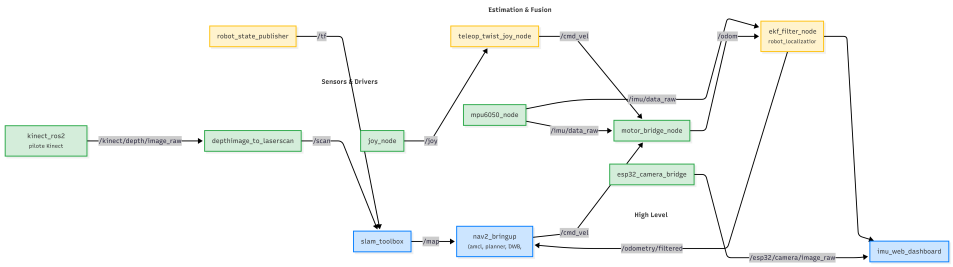


Figure 2.2. Graphe simplifié des nœuds et topics ROS 2. Couleurs : vert = pilotes, gris-bleu = estimation d'état, bleu = planification/cartographie.

2.4 Synthèse des topics principaux

Le tableau 2.1 résume les topics ROS 2 de communication. La liste exhaustive est fournie en annexe A.

Table 2.1. Topics ROS 2 principaux.

Topic	Type	Publié par	Souscrit par
`/cmd_vel`	geometry_msgs/Twist	teleop_twist_joy, Nav2	motor_bridge_node
`/odom`	nav_msgs/Odometry	motor_bridge_node	ekf_filter_node
`/odometry/filtered`	nav_msgs/Odometry	ekf_filter_node	Nav2, imu_web_dashboard
`/imu/data_raw`	sensor_msgs/Imu	mpu6050_node	motor_bridge_node, ekf_filter_node, dashboard
`/scan`	sensor_msgs/Laser-Scan	depthimage_to_laser-scan (Kinect)	slam_toolbox, Nav2 costmaps
`/map`	nav_msgs/Occupancy-Grid	slam_toolbox / map_server	Nav2, RViz2, m-explore
`/tf`, `/tf_static`	tf2_msgs/TFMessage	robot_state_publisher, ekf, slam_toolbox	Nav2, RViz2
`/joy`	sensor_msgs/Joy	joy_node	teleop_twist_joy_node, dashboard
`/esp32/camera/image_raw`	sensor_msgs/Image	esp32_camera_bridge	(consommateur optionnel)

CHAPITRE 3

Conception Mécanique et Électronique

3.1 Architecture différentielle

EsiBot adopte une architecture *differential drive* : deux roues motrices indépendantes plus une roue folle assurant la stabilité statique, la direction résultant uniquement de la différence de vitesse entre les deux roues. Les fichiers Computer-Aided Design (CAO) (CAD)/STL du châssis constituent un livrable séparé (Livrable 1) non inclus dans le code analysé ; ce chapitre s'appuie donc sur le modèle URDF (`esibot_description`), source de vérité utilisée par ROS 2 pour les transformées et les costmaps, complété par les constantes électroniques relevées dans le code de commande.

3.2 Modèle géométrique

Table 3.1. Paramètres géométriques et inertiels déclarés dans le modèle URDF.

Élément	Valeur	Remarque
Châssis (`base_link`)	$0.21 \times 0.14 \times 0.03$ m	masse 2.0 kg
Rayon de roue (`wheel_radius`)	0.025 m	largeur 0.027 m, masse 0.2 kg/roue
Empattement (`wheel_y` $\times 2$)	0.14 m	distance latérale entre roues
Module caméra (`camera_link`)	$0.073 \times 0.278 \times 0.073$ m	masse 0.56 kg, $x = -0.105$ m (arrière)

Le modèle déclare les repères optiques caméra (`camera_depth_frame`, `camera_rgb_optical_frame`, rotation $-\pi/2$) ; un `static_transform_publisher` relie `camera_depth_frame` au repère natif du pilote Kinect `kinect_depth` par une transformée identité.

Limitation

Une révision des paramètres géométriques (notamment le rayon des roues et l'empattement) est nécessaire pour garantir une parfaite cohérence entre le modèle URDF et le code de commande bas-niveau, afin d'optimiser la précision de l'odométrie.

3.3 Cinématique différentielle et odométrie

La conversion `/cmd_vel` $\rightarrow$ vitesses de roue, implémentée dans `motor_bridge_node.py` (`twist_to_wheel_commands`) et répliquée dans le tableau de bord (`imu_web_dashboard.py`), suit le modèle classique :

$$v_{\mathrm{g}} = v - \omega \cdot \frac{L}{2}, \quad v_{\mathrm{d}} = v + \omega \cdot \frac{L}{2} \tag{3.1}$$

et inversement pour la reconstruction d'odométrie : $v = (v_{\mathrm{g}} + v_{\mathrm{d}})/2$, $\omega = (v_{\mathrm{d}} - v_{\mathrm{g}})/L$ (équation 3.2), où L est l'empattement.

$$v = \frac{v_{\mathrm{g}} + v_{\mathrm{d}}}{2}, \qquad \omega = \frac{v_{\mathrm{d}} - v_{\mathrm{g}}}{L} \tag{3.2}$$

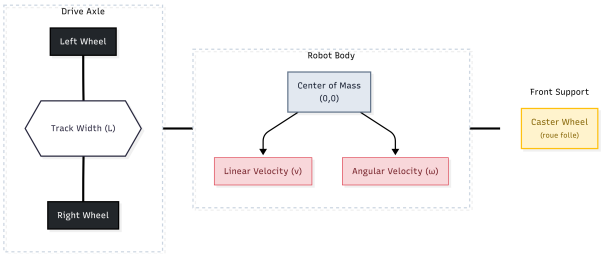


Figure 3.1. Modèle cinématique du robot différentiel **EsiBot** (vue de dessus).

Chaque roue est équipée d'un encodeur compté par interruption GPIO, anti-rebond logiciel 2 ms. Le paramètre `encoder_cpr` (12 par défaut) est doublé par comptage sur les deux fronts (`encoder_count_both_edges=True`), donnant **24 impulsions/tour**. La vitesse de rotation mesurée est calculée à chaque cycle de contrôle (`control_hz`, 10 Hz par défaut) par :

$$\mathrm{RPM}_{\text{mesuré}} = \frac{N_{\text{impulsions}}}{\mathrm{CPR}_{\text{effectif}}} \times \frac{60}{\Delta t} \tag{3.3}$$

soit $360°/24 = 15°$ par impulsion et, avec le rayon de 0.033 m du calibrateur, ≈**8.6 mm**/impulsion ($2\pi \times 0.033/24$).

3.4 Câblage électronique

Le cahier des charges (Livrable 2) exige un schéma Fritzing/KiCad, absent de l'archive fournie ; le tableau 3.2 a donc été **reconstruit par rétro-ingénierie** depuis les constantes GPIO du code Python. À la différence du cahier des charges, qui prévoyait que l'ESP32-CAM pilote le pont en H, le servo et le HC-SR04, l'implémentation réelle concentre toute l'électronique temps-réel sur les broches GPIO du Raspberry Pi 4.

Table 3.2. Broches GPIO du Raspberry Pi 4 (BCM), reconstruites depuis le code source.

Fonction	GPIO	Source / nœud
ENA / IN1 / IN2 (moteur g.)	12 / 5 / 6	`motor_bridge_node`, `motor_calibrator`
Encodeur gauche	26	`motor_bridge_node` (`pin_left_encoder`)
ENB / IN3 / IN4 (moteur d.)	13 / 21 / 20	`motor_bridge_node`, `motor_calibrator`
Encodeur droit	27	`motor_bridge_node` (`pin_right_encoder`)
I2C SDA/SCL (bus 1)	2 / 3	`mpu6050_node` (MPU6050 @ 0x68)
HC-SR04 SIG (trig/écho)	23	`ultrasonic_scan_node` (déprécié)
Servo SG90 radar	18	`ultrasonic_scan_node` (déprécié)

Dans la branche `working_build`, le câblage logiciel du calibrateur et celui du nœud de production sont cohérents : les deux utilisent les broches BCM 26 et BCM 27 pour les encodeurs gauche et droit. La calibration PWM→RPM peut donc alimenter directement `pwm_mapper.py`, sous réserve que le rayon de roue et l'empattement utilisés par l'odométrie soient contrôlés expérimentalement (voir chapitre 8).

Le pilotage moteur utilise un double pont en H **L298N** en PWM logiciel (`RPi.GPIO`, 500 Hz), rapport cyclique borné à $[15\,\%, 100\,\%]$ (`min/max_duty_cycle`) pour compenser la zone morte mécanique. Le MPU6050 (I2C bus 1, adresse `0x68`) est configuré au démarrage (réveil, diviseur d'échantillonnage `SMPLRT_DIV=0x07`, filtre passe-bas `DLPF_CFG=0x03`, $\pm 2g/\pm 250°$/s) et publie à **50 Hz**. Le Kinect (v1, ère Xbox 360), capteur de cartographie principal, est relié en **USB** (non GPIO) et piloté par le paquet communautaire `kinect_ros2` (`libfreenect`).

3.5 Serveur de caméra (ESP32-CAM)

Le module ESP32-CAM (`AI_THINKER`) sert exclusivement de serveur vidéo MJPEG WiFi : le firmware `CameraServerArduinoIDE.ino` initialise le capteur OV2640 (`UXGA` si PSRAM, sinon `SVGA`) et expose le flux sur `/stream`. Aucune broche GPIO de l'ESP32-CAM ne pilote d'actionneur — l'essentiel de ses broches étant réservé à l'interface caméra parallèle — une contrainte matérielle probablement à l'origine du report de la commande moteur vers le Raspberry Pi 4. Les identifiants WiFi (SSID/mot de passe) sont **codés en dur** dans le firmware, sans portail captif ni provisioning (chapitre 8).

3.6 Alimentation

Le schéma d'architecture (figure 2.1) identifie un bloc « Power Management » alimentant séparément la couche actionneurs (L298N) et la couche de calcul (Raspberry Pi 4, ESP32-CAM). Aucun nœud de supervision de batterie (`/battery_state`, prévu au cahier des charges) n'a été retrouvé dans le code : la télémétrie de batterie n'est, à ce stade, pas exposée par le logiciel embarqué.

CHAPITRE 4

Commande Bas-Niveau et Fusion de Capteurs

4.1 Contrôle moteur (`motor_bridge_node`)

Ce nœud (paquet `esibot_hardware`) constitue le cœur de la commande temps-réel du robot. Il joue, sur le Raspberry Pi 4, le rôle initialement dévolu au binôme ESP32/`esibot_driver` du cahier des charges : réception de `/cmd_vel`, asservissement de vitesse de roue, lecture des encodeurs, et publication de l'odométrie brute.

4.1.1 Boucle de contrôle

Le nœud exécute une boucle périodique à `control_hz` (10 Hz par défaut) qui, à chaque cycle :

1. lit les compteurs d'impulsions accumulés depuis le dernier cycle pour chaque roue ;
2. calcule la vitesse de rotation mesurée en rpm (équation 3.3) ;
3. applique un correcteur PID (un par roue) pour rapprocher la vitesse mesurée de la consigne issue de la cinématique différentielle (équation 3.1) ;
4. convertit la sortie du correcteur en rapport cyclique PWM via `pwm_mapper.py` ;
5. applique une correction de cap optionnelle à partir de l'IMU (asservissement de lacet) ;
6. publie l'odométrie intégrée sur `/odom` et l'état des compteurs sur les topics de diagnostic (`/encoder_ticks\protect\penalty\z@/left`, `/encoder_ticks\protect\penalty\z@/right`).

4.1.2 Correcteur PID par roue

Chaque roue dispose de son propre correcteur PID indépendant, avec gains par défaut K_p, K_i, K_d paramétrables via le serveur de paramètres ROS 2 (et ajustables en exploitation sans recompilation). La sortie du correcteur est saturée et un mécanisme anti-*windup* limite l'accumulation du terme intégral lorsque la commande est saturée, évitant un dépassement (*overshoot*) lors des changements brusques de consigne.

Boucle de sécurité

Le nœud implémente une minuterie de *watchdog* : si aucun message `/cmd_vel` n'est reçu pendant une durée `cmd_vel_timeout` (typiquement quelques centaines de millisecondes), les deux moteurs sont automatiquement coupés. Ce mécanisme protège le robot en cas de perte de communication réseau avec le nœud de téléopération ou de Nav2.

4.2 Conversion vitesse de roue et PWM

Le module `pwm_mapper.py` convertit une vitesse linéaire de roue souhaitée (en m/s) en rapport cyclique PWM normalisé. Dans `working_build`, cette conversion n'est plus un simple étalonnage théorique : elle charge le fichier `~/.esibot/pwm_rpm_map.json`, généré sur le robot, et applique une régression affine séparée pour chaque roue et chaque sens de marche. La conversion combine :

- une zone morte mécanique (`min_duty_cycle`, 15 % par défaut et 22 % dans certains lancements sans calibration) ;
- des coefficients a/b calibrés pour `left/right` et `forward/reverse`, afin de compenser les différences de rendement des deux moteurs ;
- un plafonnement à `max_pwm`/`max_rpm` issus de la calibration, ce qui évite de commander une zone de saturation ou de bruit mécanique.

4.3 Outil de calibration (`motor_calibrator.py`)

Un script de calibration autonome (hors graphe ROS 2, exécuté en ligne de commande) caractérise empiriquement la relation entre rapport cyclique PWM et vitesse de rotation réelle (en rpm) pour chaque moteur. Il balaie les rapports cycliques de 15 % à 100 % par pas de 5 %, mesure deux échantillons d'une seconde par point, conserve la médiane, puis arrête le balayage dès qu'une chute de RPM indique une saturation ou une mesure instable. Le résultat est sauvegardé dans `~/.esibot/pwm_rpm_map.json` avec le rayon de roue utilisé (0.033 m), la fréquence PWM (500 Hz) et les coefficients avant/arrière de chaque roue. Cet outil est essentiel pour limiter l'asymétrie mécanique des moteurs, mais il ne suffit pas seul à garantir une odométrie correcte si le rayon de roue réel, l'empattement et les paramètres SLAM ne sont pas recalés ensemble (chapitre 8).

4.4 Centrale inertielle et stabilisation de cap

Le nœud `mpu6050_node` (paquet `esibot_sensors`) démarre par une calibration stationnaire du biais gyroscopique sur 100 échantillons, puis publie en continu (50 Hz) :

- `/imu/data_raw` (`sensor_msgs/Imu`) — accélérations linéaires (a_x, a_y, a_z) et vitesses angulaires $(\omega_x, \omega_y, \omega_z)$ brutes ;
- `/imu/temperature` (`sensor_msgs/Temperature`) — température interne du capteur, utile au diagnostic thermique.

`motor_bridge_node` souscrit à `/imu/data_raw` pour appliquer une correction de lacet en boucle fermée lorsque le robot avance et qu'aucune rotation volontaire n'est commandée. Le correcteur compense une vitesse de lacet résiduelle mesurée par l'IMU, avec une bande morte de 0.02 rad/s et une correction maximale de 40 RPM par roue.

4.5 Fusion de données (Filtre EKF)

L'odométrie brute (`/odom`, issue des seuls encodeurs) et les données inertielles (`/imu/data_raw`) sont fusionnées par un EKF fourni par le paquet standard `robot_localization` (nœud `ekf_filter_node`), configuré en mode bidimensionnel (`two_d_mode: true`). Le fichier `ekf_odom_imu.yaml` de `working_build` est volontairement conservateur : il fusionne les vitesses d'odométrie $(v_x, v_y, \dot{\theta})$ et uniquement la vitesse de lacet de l'IMU. Il n'injecte ni orientation absolue ni accélération linéaire, car le MPU6050 ne publie pas d'orientation estimée et son accélération brute contient gravité et biais résiduel. La sortie `/odometry/filtered` fournit donc une odométrie locale plus stable pour SLAM/Nav2, tandis que `slam_toolbox` ou Adaptive Monte Carlo Localization (AMCL) gardent la responsabilité de la correction globale `map` → `odom`.

CHAPITRE 5

Cartographie et Navigation Autonome

5.1 Pipeline Kinect vers SLAM

Le système repose sur un capteur de profondeur **Kinect** pour générer la cartographie de l'environnement. Les données de profondeur extraites sont converties en balayage laser 2D. Cette approche permet de fournir la structure de message standard `sensor_msgs/LaserScan` attendue par l'algorithme SLAM. Le fichier `mapping.launch.py` lance et orchestre l'ensemble de cette chaîne (figure 5.1).

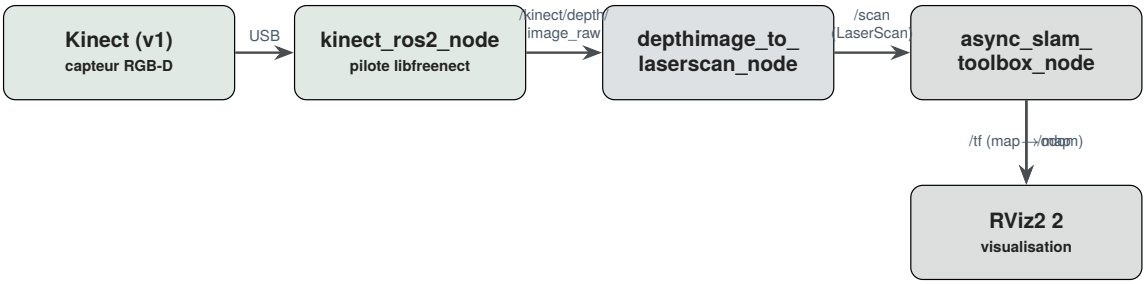


Figure 5.1. Pipeline de cartographie tel qu'orchestré par `mapping.launch.py`.

Le pilote `kinect_ros2` (espace de noms `kinect`, basé sur `libfreenect`) expose `/kinect/depth/image_raw` et `/kinect/depth/camera_info`. Un `static_transform_publisher` relie `camera_depth_frame` (URDF) au repère natif `kinect_depth` par une transformée identité. Le nœud `depthimage_to_laserscan_node` projette ensuite une bande horizontale de l'image de profondeur en balayage laser synthétique (tableau 5.1).

Table 5.1. Paramètres de `depthimage_to_laserscan_node` (`mapping.launch.py`).

Paramètre	Valeur	Signification
`scan_height`	10 px	bande de l'image de profondeur moyennée par balayage
`scan_time`	0.033 s	période nominale (≈30 Hz)
`range_min` / `max`	0.35 / 4.0 m	portée exploitable du Kinect
`output_frame`	`camera_depth_frame`	repère TF du `LaserScan`

Champ de vision réduit

Le Kinect couvre $\approx 57°$ horizontalement contre 180–360 degrés pour un radar rotatif ou un LiDAR panoramique prévu : les zones latérales/arrière restent aveugles à un instant donné, nécessitant une rotation active du robot pendant la cartographie pour compenser.

5.2 Prétraitement de profondeur `depth_cleanup`

La branche `working_build` ajoute le paquet `depth_cleanup`, une bibliothèque C++/OpenCV indépendante de ROS 2 et de `libfreenect`. Son rôle est de nettoyer les trous de profondeur Kinect tout en conservant une image métrique `CV_16UC1` en millimètres, utilisable ensuite par un nœud ROS 2 qui publierait une `sensor_msgs/Image` synchronisée avec `CameraInfo`. Le mode retenu pour la géométrie est `applyKinectSmoothingPipeline()` :

- les pixels nuls sont traités comme invalides ;
- un voisinage 3×3 remplit les petits trous par mode quantifié en classes de 10 mm ;
- les trous restants sont comblés par `cv::INPAINT_TELEA` à 20 % de résolution pour limiter le coût CPU ;
- les pixels valides originaux restent inchangés, ce que vérifient les tests unitaires du paquet.

Le document `SLAM_INTEGRATION.md` fixe explicitement la frontière d'intégration : `slam_toolbox` ne consomme pas l'image de profondeur directement, mais le `LaserScan` produit par `depthimage_to_laserscan`. À l'état actuel du lancement `mapping.launch.py`, le remappage pointe encore vers `/kinect/depth/image_raw`. L'étape restante consiste donc à insérer un adaptateur ROS 2 publiant l'image nettoyée `16UC1` et son `CameraInfo`, puis à remapper `depthimage_to_laserscan` vers ce topic nettoyé.

5.3 Pourquoi le Kinect remplace le radar ultrasonique

Le chemin ultrasonique reste présent dans `ultrasonic_scan_node`, mais il n'est pas adapté à la conduite autonome rapide. La configuration de `working_build` effectue un balayage de seulement $-30°$ à $+30°$ par pas de $5°$, prend cinq mesures par angle, attend 0.35 s de stabilisation servo, et publie environ 0.07 balayage par seconde. Un tel capteur donne une distance scalaire par direction, séquentielle et fortement dépendante de l'orientation instantanée du servo : une chaise, un pied de table, un obstacle bas ou une surface inclinée peuvent être manqués ou interprétés au mauvais endroit entre deux mises à jour de costmap.

L'article de Kamarudin et al. sur l'analyse du Kinect pour le SLAM 2D, référencé en annexe A.3, rappelle qu'un capteur Kinect utilisé en SLAM 2D reste limité par son champ de vision et sa portée, mais que sa donnée de profondeur 3D offre une base plus riche pour détecter des objets de formes et tailles variables. Pour **EsiBot**, cela transforme le problème : au lieu de construire une costmap à partir d'un faisceau ultrasonique lent, on extrait une coupe LaserScan cohérente depuis une scène de profondeur complète et mise à jour à cadence vidéo. Le radar ultrasonique garde donc un intérêt de diagnostic ou de sécurité locale lente, mais le Kinect est le capteur cohérent pour alimenter `slam_toolbox`, Nav2 et l'exploration autonome.

5.4 Configuration de `slam_toolbox`

`slam_toolbox_esibot_kinect.yaml` configure `async_slam_toolbox_node` en mode `mapping` (solveur Ceres, `SPARSE_NORMAL_CHOLESKY`). Le tableau 5.2 résume les paramètres clés.

Table 5.2. Paramètres clés de `slam_toolbox` pour **EsiBot**.

Paramètre	Valeur	Rôle
`resolution`	0.05 m	taille de cellule de la grille d'occupation
`min/max_laser_range`	0.35 / 4.0 m	cohérent avec `depthimage_to_laserscan`
`minimum_time_interval`	0.5 s	intervalle minimal entre balayages traités
`minimum_travel_distance/heading`	0.15 m / 0.15 rad	filtre anti-suréchantillonnage
`do_loop_closing`	`true`	fermeture de boucle activée (rayon 3.0 m)
`transform_publish_period`	0.02 s	publication `map`→`odom` (≈50 Hz)

`use_map_saver: true` permet la sauvegarde à la demande (`map_saver_cli`) en paire `map.pgm`/`map.yaml`, ensuite consommée par Nav2.

Limitation

La carte actuelle de `esibot_navigation2` (résolution 0.05 m, origine $(-10, -10, 0)$) correspond, après inspection visuelle, à la carte hexagonale d'exemple par défaut TurtleBot3/Nav2, et non à une carte réelle capturée par le Kinect d'**EsiBot**. La chaîne SLAM → sauvegarde → rechargement Nav2 restait donc à valider sur le terrain au moment de la rédaction (chapitre 7).

La configuration RViz2 `esibot_slam_map.rviz` constitue l'outil de visualisation principal du projet (carte, balayage `/scan`, modèle URDF, arbre TF complet), plutôt qu'une vue cartographique intégrée au tableau de bord web (chapitre 6).

5.5 Navigation Autonome avec Nav2

La navigation autonome repose sur la pile standard **Nav2** (`navigation2.launch.py`, paquet `esibot_navigation2`), chargeant la carte statique, le fichier `esibot.yaml`, et orchestrant localisation, costmaps, planificateur, contrôleur, comportements, lisseur de vitesse et moniteur de collision.

La localisation utilise AMCL (filtre particulaire, 500–2000 particules, modèle `likelihood_field`, `DifferentialMotionModel`, pose initiale non définie automatiquement). Les costmaps locale (fenêtre 3×3 m) et globale partagent une résolution de **0.05 m/cellule**, un rayon de robot de **0.2 m**, et une inflation de **0.35 m** (facteur 5.0). Le planificateur global `NavfnPlanner` (Dijkstra, tolérance 0.5 m) alimente le contrôleur local Dynamic Window-Based local planner (DWB), dont les bornes sont résumées au tableau 5.3.

Table 5.3. Bornes cinématiques DWB et AMCL (extrait de `esibot.yaml`).

Paramètre	Valeur	Unité
Vitesse linéaire min. / max. (DWB)	0.2 / 0.3	m/s
Vitesse angulaire max. (DWB)	0.5	rad/s
Accélération / décélération linéaire	3.0 / -2.5	m/s^2
Tolérance position du but (`FollowPath`)	0.05	m
Vitesse max. (lisseur de vitesse)	0.5 / 2.5	m/s / rad/s

Les critères de score DWB (`PathAlign` 32, `GoalDist` 24, `BaseObstacle` 0.02, `RotateToGoal` 32) privilégient nettement le respect du chemin global sur la vitesse, cohérent avec un robot de faible gabarit en intérieur. Le serveur de comportements expose `Spin`, `BackUp`, `DriveOnHeading`, `Wait`, `AssistedTeleop` ; un `collision_monitor` surveille un disque d'arrêt (rayon 0.1 m), un polygone de ralentissement et un polygone d'approche (horizon 2 s) avant transmission finale à `/cmd_vel`.

Comme indiqué en section 5.4, la carte chargée par défaut est une carte d'exemple générique ; la configuration Nav2 elle-même est complète et cohérente avec le pipeline Kinect, mais sa validation expérimentale sur une carte réelle restait à conduire (chapitre 7).

5.6 Exploration autonome

La branche `working_build` ajoute également `src/m-explore-ros2` comme dépendance Git. Ce module correspond à la couche d'exploration par frontières : il se place au-dessus de la carte `/map`, de l'arbre TF et de Nav2, sélectionne des frontières entre zones connues et inconnues, puis envoie des objectifs successifs au navigateur. Aucun fichier de lancement `esibot_*` ne l'orchestre encore avec le bringup, le Kinect et Nav2 ; son intégration opérationnelle doit donc vérifier trois conditions avant démonstration : sous-module initialisé dans le workspace, topics `/map`/`/tf` stables, et Nav2 actif avec une carte ou un SLAM en ligne.

CHAPITRE 6

Interface Utilisateur et Mise en Œuvre

6.1 Téléopération par manette Xbox

La téléopération repose sur une manette sans fil (compatible Xbox), pilotée par `joy_node`/`teleop_twist_joy` (`joystick_drive.launch.py`), et non le `teleop_twist_keyboard` suggéré au cahier des charges.

Table 6.1. Configuration de la téléopération manette (`xbox_teleop.yaml`).

Paramètre	Valeur	Remarque
`require_enable_button`	`false`	aucune touche « homme-mort » requise
Axe linéaire / angulaire	axe 1 (éch. 0.25) / axe 3 (éch. 2.4)	sticks → `/cmd_vel`
Zone morte / répétition	0.08 / 20 Hz	`joy_node`

L'absence de bouton d'activation obligatoire signifie que toute déviation accidentelle des manches se traduit immédiatement par un mouvement, sans touche à maintenir — une marge de sécurité réduite par rapport à un dispositif « homme-mort » classique (chapitre 8). `joystick_drive.launch.py` démarre conjointement `joy_node`, `teleop_twist_joy_node` et `motor_bridge_node` (vitesse angulaire max. relevée à 3.0 rad/s, rapport cyclique minimal 22 %).

6.2 Fusion EKF — détail de configuration

`ekf_odom_imu.yaml` (`ekf_fusion.launch.py`) configure `ekf_filter_node` à **30 Hz**, `two_d_mode` : de l'odométrie (`/odom`), seules les *vitesses* $(v_x, v_y, \dot{\theta})$ sont fusionnées ; de l'IMU (`/imu/data_raw`), seul $\dot{\theta}$ est retenu. Cette configuration évite d'intégrer l'accélération brute du MPU6050 ou une orientation que le nœud ne fournit pas. Le `world_frame` reste `odom`, laissant la correction `map`→`odom` à `slam_toolbox`/AMCL — une séparation propre entre estimation locale (EKF) et globale (SLAM/AMCL).

6.3 Tableau de bord web de télémétrie

Le nœud `imu_web_dashboard` (`esibot_dashboard`) expose un serveur HTTP autonome (`http.server`, sans dépendance externe), diffusant une page HTML/JS à page unique, sensiblement différente du tableau de bord du cahier des charges (qui prévoyait une passerelle WebSocket `rosbridge_suite` avec carte, vidéo et boutons de téléopération) — l'implémentation réelle se concentre sur la **télémétrie**.

Table 6.2. Widgets du tableau de bord `imu_web_dashboard`.

Widget	Contenu
Orientation	roulis/tangage (intégration accéléromètre), horizon artificiel `<canvas>`, mise à zéro (`POST /api/zero`)
Odométrie	position/cap/vitesses (`/odometry/filtered`), trajectoire (1200 pts)
Manette & commande	axes/boutons `/joy` et conversion `/cmd_vel` → vitesses de roue (équation 3.1)

L'interface interroge en boucle `GET /api/imu` (état JSON consolidé, protégé par verrou pour la cohérence entre thread ROS 2 et thread HTTP). Le tableau de bord **n'affiche ni la carte SLAM ni le flux vidéo**, deux fonctionnalités du Livrable 5 (chapitre 8).

6.4 Flux vidéo ESP32-CAM

Le pont `esp32_camera_bridge` (`mjpeg_image_publisher`) se connecte en client HTTP au flux MJPEG de l'ESP32-CAM, extrait chaque trame JPEG (marqueurs `0xFFD8`/`0xFFD9`), la décode avec OpenCV et republie sur `/esp32/camera/image_raw` via `cv_bridge`, avec reconnexion automatique (2 s). Ce flux est consommable mais non consommé par un nœud de vision dédié — aucun `vision_node` (détection ligne/obstacles, prévu au cahier des charges) n'a été retrouvé dans le code livré.

6.5 Mise en Œuvre et Procédures d'Exploitation

Le dépôt fournit plusieurs scripts shell (`scripts/`) qui illustrent la séparation bas-niveau/haut-niveau, contrairement au test T1 du cahier des charges qui suppose un unique `esibot_full.launch.py`.

Table 6.3. Scripts de démarrage disponibles dans `scripts/`.

Script	Rôle	Particularité
`start_esibot.sh`	lancement manuel du calibrateur	script de développement minimal
`start_esibot_all.sh`	démarrage production (téléop, IMU, EKF, dashboard)	exige une calibration valide ; radar commenté dans le script
`start_esibot_all_no_calib.sh`	variante sans vérification	démarre le radar ultrasonique en plus de la téléopération
`start_esibot_all_no_calib_no_imu.sh`	bringup sans calibration obligatoire	IMU, EKF et dashboard activables par variables d'environnement
`start_esibot_keyboard_autonomous.sh`	mode commande clavier/autonomie	pas de joystick ; attend un éditeur de `/cmd_vel` externe
`read_pwm_pins.py`, `read_wheel_encoders.py`	diagnostic GPIO	lecture directe des PWM et des ticks encodeurs

Aucun script ne démarre le pipeline Kinect/SLAM

Les scripts shell se limitent à la couche « commande bas-niveau + télémétrie » (chapitre 4). Le pipeline de cartographie (`mapping.launch.py`) et Nav2 (`navigation2.launch.py`) doivent être démarrés *séparément*, typiquement depuis le PC opérateur (couche haut-niveau, figure 2.1) — reflet direct du découpage en trois couches (chapitre 2).

Avant tout démarrage via `start_esibot_all.sh`, le script vérifie la validité d'un fichier de calibration JSON (`~/.esibot/pwm_rpm_map.json`, généré par `motor_calibrator`, section 4.3), avec coefficients positifs exigés pour les 4 combinaisons {gauche,droite}×{avant,arrière} — un garde-fou bloquant le démarrage en cas de calibration absente ou invalide.

```
./scripts/start_esibot_all.sh        # teleop + IMU + EKF + dashboard (RPi4)

ros2 launch esibot_slam mapping.launch.py start_rviz:=true        # cartographie
ros2 run nav2_map_server map_saver_cli -f ~/esibot_ws/maps/salle_test
ros2 launch esibot_navigation2 navigation2.launch.py \
    map:=$HOME/esibot_ws/maps/salle_test.yaml use_rviz:=true      # navigation
```

Listing 6.1. Séquence complète — démarrage bas-niveau, puis cartographie et navigation.

Le workspace (`~/esibot_ws`) regroupe les paquets `esibot_*`, la bibliothèque `depth_cleanup`, les dépendances tierces `kinect_ros2`, `esp32_camera_bridge` et `m-explore-ros2`, en complément des paquets standards ROS 2 Humble (`slam_toolbox`, `nav2_bringup`, `robot_localization`, `depthimage_to_laserscan`, `joy`, `teleop_twist_joy`, `cv_bridge`). Le fichier `START_EVERYTHING.md` de `working_build` documente la séquence de build, les commandes de démarrage manuel, les vérifications de topics et la sauvegarde de carte.

CHAPITRE 7

Résultats Mesurés et Paramètres Système

7.1 Valeurs de configuration

Les paramètres de fonctionnement déclarés dans l'architecture dictent le comportement global du robot. Le tableau 7.1 regroupe les caractéristiques clés dérivées de la configuration du système.

Table 7.1. Grandeurs dérivées des paramètres de configuration.

Grandeur	Valeur	Calcul / source
Résolution encodeur effective	24 imp./tour	`encoder_cpr=12`, comptage double front
Fréquences boucle commande / IMU / EKF	10 / 50 / 30 Hz	`control_hz`, `mpu6050_node`, `ekf_odom_imu.yaml`
Calibration biais gyroscope	100 échantillons	`gyro_calibration_samples`
Cadence `/scan` brute / intégrée SLAM	≈30 Hz / ≤2 Hz	`scan_time=0.033 s` ; intervalle=0.5 s
Cadence radar ultrasonique	≈0.07 Hz	`ultrasonic_radar.yaml`
Vitesse linéaire/angulaire max. (téléopération)	0.25 m/s / 2.4 rad/s	`scale_linear.x`, `scale_angular.yaw`
Vitesse linéaire min./max. (Nav2/DWB)	0.2 / 0.3 m/s	`esibot.yaml`
Tolérance position du but (`FollowPath`)	0.05 m	`esibot.yaml`
Résolution carte SLAM/costmaps	0.05 m/cellule	cohérent `slam_toolbox`/Nav2

Asymétrie de vitesse

La vitesse angulaire maximale en téléopération manuelle (2.4 rad/s) est établie plus haute que la vitesse autonome pour garantir une forte réactivité humaine.

7.2 Plan de Validation

La validation physique de la plateforme repose sur les scénarios du tableau 7.2, qui vérifient la fiabilité de chaque couche.

Table 7.2. Plan de validation terrain.

Scénario	Procédure	Critère attendu
Démarrage bas-niveau	Lancer moteurs, MPU6050, EKF et dashboard	topics `/odom`, `/imu/data_raw`, `/odometry/filtered` stables
Cohérence odométrique	Parcourir 2 m en ligne droite après calibration PWM	dérive latérale mesurée et reproductible
Pipeline Kinect	Comparer `/scan` à des distances murales connues	erreur de portée bornée et sans trous persistants
Cartographie	Construire une carte Kinect d'une zone 3×3 m	murs continus, fermeture de boucle plausible
Navigation	Envoyer trois buts Nav2 successifs sur carte sauvegardée	arrivée sans collision et arrêt contrôlé
Exploration	Initialiser `m-explore-ros2` avec SLAM/Nav2 actifs	objectifs frontière émis sans conflit TF

Table 7.3. État consolidé des validations dans le rapport.

Élément	**État actuel**	**Mesure restante**
Commande moteur	implémentée et calibrable	courbe PWM/RPM finale sur robot assemblé
EKF odom+IMU	configuré conservativement	dérive avant/après fusion sur trajectoire répétée
Kinect → `/scan`	lancé par `mapping.launch.py`	comparaison raw/cleaned depth après intégration `depth_cleanup`
Nav2	configuration complète	essais multi-buts sur carte EsiBot réelle
Exploration	dépendance ajoutée	orchestration par launch et test de frontières

CHAPITRE 8

Analyse Critique, Limitations et Écarts

Ce chapitre distingue les écarts assumés du projet et les limites qui demandent encore une validation expérimentale. L'état actuel reste prometteur : l'architecture logicielle est cohérente, les nœuds principaux existent, et les choix Kinect/Nav2/EKF donnent une base techniquement solide. Les points ci-dessous relèvent surtout de la calibration fine nécessaire pour transformer une intégration fonctionnelle en produit démontrable de façon répétable.

8.1 Compromis architecturaux

Au cours du développement, plusieurs ajustements ont été opérés par rapport aux concepts initiaux :

Table 8.1. Synthèse des évolutions architecturales.

Aspect	Implémentation finale
Commande moteurs/servo	Centralisée sur le Raspberry Pi 4 ; l'ESP32-CAM est dédié au flux vidéo.
Capteur de cartographie	Kinect v1 via `kinect_ros2` et `depthimage_to_laserscan`, avec `depth_cleanup` prêt pour un flux nettoyé.
Radar ultrasonique	Conservé dans le code pour test ou secours lent, mais non retenu comme capteur principal de conduite autonome.
Exploration	`m-explore-ros2` référencé par la branche, encore à orchestrer dans un lancement EsiBot complet.
Passerelle web	Serveur HTTP Python spécifique pour la télémétrie, plus simple que la passerelle WebSocket prévue initialement.

Ces écarts ne dégradent pas la cohérence du produit. Ils clarifient au contraire une architecture plus réaliste : la commande temps réel reste sur le Raspberry Pi, tandis que la perception et la planification lourdes restent déportables vers le PC opérateur.

8.2 Calibration croisée odométrie–SLAM

La limitation la plus profonde n'est pas un module absent, mais l'accord numérique entre les topics qui décrivent le même mouvement. Une calibration PWM→RPM correcte ne suffit pas si le rayon de roue, l'empattement, la correction de lacet, la fusion EKF et les seuils de `slam_toolbox` ne convergent pas vers le même déplacement réel.

Table 8.2. Paramètres à calibrer conjointement.

Chaîne	Paramètres sensibles	Symptôme si mal calibré
Roues → `/odom`	rayon 0.033 m du calibrateur, `wheel_base`=0.16 m, CPR effectif	distance ou rotation intégrée biaisée
URDF/TF	rayon 0.025 m dans le modèle, empattement visuel 0.14 m	incohérence entre modèle RViz2, costmaps et odométrie
IMU → EKF	biais gyroscope, seuils de lacet, `imu0_config`	robot corrigé trop fort ou pas assez en ligne droite
`/scan` → carte	`minimum_travel_distance`, `minimum_travel_heading`, portée Kinect	carte étirée, murs doublés ou boucles fermées instables
Nav2 → `/cmd_vel`	limites DWB, vitesse min. 0.2 m/s, accélérations	oscillations ou trajectoires trop agressives pour les moteurs

La méthode recommandée est manuelle et basée sur des sacs ROS 2 courts : enregistrer `/cmd_vel`, `/odom`, `/odometry/filtered`, `/scan`, `/tf` et `/map` sur une ligne droite, une rotation sur place et un carré de référence ; comparer la distance réelle, la trajectoire odométrique et la correction `map`→`odom` ; ajuster ensuite le rayon, l'empattement et les gains de lacet avant de modifier les seuils de `slam_toolbox`. Cette étape demande une analyse humaine car un même défaut visible sur la carte peut venir d'une roue mal étalonnée, d'un lacet IMU biaisé, d'un scan Kinect bruité ou d'un seuil de mise à jour trop élevé.

8.3 Capteurs de perception

Le passage du radar ultrasonique au Kinect améliore fortement la conduite autonome, mais ne transforme pas le système en LiDAR. Les limites restantes sont connues et calibrables :

- **Radar ultrasonique** — le balayage actuel est lent (≈0.07 Hz), étroit ($60°$) et séquentiel. Il ne fournit pas une observation assez dense pour une costmap dynamique ; il doit rester un outil de test ou un secours très local.
- **Kinect** — le champ horizontal d'environ $57°$, la portée utile 0.35–4.0 m et la sensibilité à la lumière infrarouge imposent une rotation active du robot et un usage intérieur. Ces limites sont acceptables pour le scénario de démonstration, car le Kinect donne malgré tout une scène de profondeur bien plus exploitable qu'un unique faisceau ultrasonique.
- **`depth_cleanup`** — la bibliothèque existe et ses tests protègent le format métrique, mais le nœud ROS 2 d'adaptation n'est pas encore inséré dans `mapping.launch.py`. Le gain sur `/scan` doit donc être mesuré avant soutenance.

8.4 Sûreté et exploitation

- **Téléopération sans bouton d'activation** — la configuration Xbox accepte une commande dès que le stick bouge. C'est pratique en essai, mais un mode de démonstration devrait réintroduire un bouton homme-mort ou un arrêt logiciel clair.
- **Supervision batterie absente** — aucun nœud ne publie `/battery_state`. Les essais longs doivent donc être surveillés hors logiciel pour éviter une coupure pendant la navigation.
- **WiFi ESP32-CAM codé en dur** — acceptable en prototype local, mais à remplacer par un mécanisme de configuration si le dépôt devient public ou réutilisé.

8.5 Performance et orchestration

L'exécution simultanée du Kinect, de `depthimage_to_laserscan`, de `slam_toolbox`, de Nav2 et de l'exploration peut dépasser le confort d'un Raspberry Pi 4 seul. La séparation en trois couches reste donc pertinente : le Raspberry Pi pilote le matériel et publie `/odom`/`/imu/data_raw`, tandis que le PC opérateur peut porter Kinect, SLAM, Nav2, RViz2 et exploration. Le point à valider n'est pas seulement la charge CPU, mais aussi la latence réseau DDS et la fraîcheur des TF, surtout lorsque l'exploration envoie des objectifs pendant la construction de carte.

8.6 Synthèse

Le produit reste optimiste dans ses résultats attendus : l'architecture choisie est meilleure que le plan ultrasonique initial pour une navigation autonome crédible. La condition principale est de traiter la calibration comme une campagne expérimentale complète, pas comme un réglage isolé du moteur gauche ou droit. Une fois cette campagne faite, les limites restantes relèvent surtout de la sécurité d'exploitation et de l'orchestration des modules haut niveau.

CHAPITRE 9

Bilan et Perspectives

9.1 Bilan technique

Le projet **EsiBot** a abouti à un robot mobile différentiel fonctionnel s'alignant sur des architectures reconnues telles que le TurtleBot3 Burger. L'utilisation d'une pile logicielle ROS 2 standard (`slam_toolbox`, Nav2, `robot_localization`) pour piloter un robot bas coût a permis de concrétiser :

- une chaîne de commande bas-niveau complète et autonome (encodeurs, PID par roue, calibration PWM→RPM outillée) entièrement portée sur Raspberry Pi 4 ;
- une fusion de capteurs EKF correctement scoper (vitesses uniquement pour l'odométrie, orientation absolue pour l'IMU), évitant le double comptage de dérive ;
- un pipeline de cartographie Kinect → `depthimage_to_laserscan` → `slam_toolbox` cohérent et correctement paramétré (bornes de portée alignées, fermeture de boucle activée) ;
- une configuration Nav2 complète (AMCL, costmaps double couche, DWB, comportements de récupération, moniteur de collision) ;
- une téléopération manette et un tableau de bord web de télémétrie fonctionnels et simples à déployer (aucune dépendance externe pour le serveur HTTP).

9.2 Plan d'action priorisé

Sur la base de l'analyse du chapitre 8, les actions correctives suivantes sont proposées par ordre de priorité décroissante :

1. **Réconcilier les constantes géométriques** (rayon de roue, empattement, affectations GPIO) entre l'URDF, `motor_bridge_node` et `motor_calibrator`, idéalement via un fichier de paramètres unique partagé.
2. **Conduire la campagne de mesures de terrain** décrite au chapitre 7 (tableaux 7.2 et 7.3), en particulier la cartographie réelle d'une zone de 3 m×3 m avec le Kinect, suivie de la validation de la navigation autonome sur cette carte — la priorité la plus urgente avant la soutenance, le scénario de démonstration du jury en dépendant directement.
3. **Implémenter la supervision de batterie** (`/battery_state`) pour sécuriser les essais et les démonstrations.
4. **Ajouter un nœud de vision minimal** (détection de ligne ou de couleur par OpenCV) consommant `/esp32/camera/image_raw`, ne serait-ce qu'à titre démonstratif, pour combler l'écart du Livrable 3/cahier des charges.
5. **Étendre le tableau de bord web** pour y intégrer, au minimum, un affichage de la carte SLAM (rendu canvas de l'`OccupancyGrid`) et un aperçu basse fréquence du flux caméra, rapprochant le Livrable 5 de sa spécification complète.
6. **Réintroduire un bouton d'activation** en téléopération manette, ou à défaut documenter explicitement ce choix de conception dans le README (Livrable 8).

9.3 Perspectives d'évolution

Au-delà des corrections immédiates, plusieurs axes d'évolution pourraient enrichir **EsiBot** dans la suite du projet ou pour de futures promotions : l'exploitation conjointe du flux couleur du Kinect (actuellement seule la profondeur est utilisée) pour de la détection visuelle complémentaire au radar ultrasonique conservé en réserve ; le retour d'une fusion à trois capteurs (Kinect + radar ultrasonique + odométrie) pour une robustesse accrue en cas de défaillance ponctuelle d'une source ; l'ajout d'une supervision énergétique complète avec estimation d'autonomie restante ; et, plus largement, la consolidation de la documentation (README, schéma électronique formel) afin de fiabiliser la reproductibilité du projet par d'autres équipes, conformément à l'esprit open-source revendiqué dès le cahier des charges initial.

9.4 Mot de conclusion

Ce rapport a documenté **EsiBot** tel qu'il a été réellement conçu et codé, plutôt que tel qu'il avait été initialement envisagé sur le papier — un choix méthodologique délibéré qui, espère l'équipe, constitue une base plus utile pour la suite du projet que la simple reproduction du cahier des charges initial. Les écarts identifiés ne sont pas des échecs mais des décisions d'ingénierie tracées et justifiables ; le plan d'action ci-dessus fixe le chemin vers une validation expérimentale complète du système avant la soutenance finale.

ANNEXE A

Annexes Techniques

A.1 Table exhaustive des topics ROS 2

Table A.1. Liste exhaustive des topics ROS 2 identifiés dans le code source.

Topic	Type	Publié par	Souscrit par
`/cmd_vel`	geometry_msgs/Twist	teleop_twist_joy_node, nav2	motor_bridge_node
`/odom`	nav_msgs/Odometry	motor_bridge_node	ekf_filter_node
`/odometry/filtered`	nav_msgs/Odometry	ekf_filter_node	nav2, dashboard
`/encoder_ticks\protect\penalty\z@(_total)\protect\penalty\z@/left,right`	Int32/Int64	motor_bridge_node	(diagnostic)
`/imu/data_raw`	sensor_msgs/Imu	mpu6050_node	motor_bridge_node, ekf, dashboard
`/imu/temperature`	sensor_msgs/Temperature	mpu6050_node	dashboard
`/scan`	sensor_msgs/LaserScan	depthimage_to_laserscan (Kinect) *ou* ultrasonic_scan_node (déprécié)	slam_toolbox, nav2
`/kinect/depth/image_raw`, `/camera_info`	Image / CameraInfo	kinect_ros2_node	depthimage_to_laserscan
`/map`	nav_msgs/OccupancyGrid	slam_toolbox	nav2, RViz2
`/tf`, `/tf_static`	tf2_msgs/TFMessage	robot_state_publisher, ekf, slam_toolbox	nav2, RViz2
`/joy`	sensor_msgs/Joy	joy_node	teleop_twist_joy_node, dashboard
`/esp32/camera/image_raw`	sensor_msgs/Image	esp32_camera_bridge	(consommateur optionnel)
`/goal_pose`	geometry_msgs/PoseStamped	RViz2	nav2 (bt_navigator)
`/global,local_costmap\protect\penalty\z@/costmap`	nav2_msgs/Costmap	nav2 (costmap servers)	RViz2, planner/controller
`/plan`	nav_msgs/Path	planner_server	controller_server, RViz2

A.2 API HTTP du tableau de bord et paquets ROS 2 du dépôt

Le serveur `imu_web_dashboard` (port 8088 par défaut) expose trois points d'accès : `GET /` (ou `/index.html`), la page HTML/JS du tableau de bord ; `GET /api/imu`, l'état JSON consolidé (IMU, odométrie, manette, `/cmd_vel`) ; et `POST /api/zero`, la remise à zéro des offsets de roulis/tangage affichés.

Table A.2. Paquets ROS 2 du workspace `Esi_Bot`.

Paquet	Rôle
`esibot_bringup`	lancements d'orchestration (téléopération, IMU, EKF, tableau de bord, radar)
`esibot_hardware`	commande moteurs/encodeurs (`motor_bridge_node`), `pwm_mapper`, `motor_calibrator`
`esibot_sensors`	`mpu6050_node`, `ultrasonic_scan_node` (déprécié)
`esibot_description`	modèle URDF/xacro, lancement `robot_state_publisher`
`esibot_slam`	lancement du pipeline Kinect → `depthimage_to_laserscan` → `slam_toolbox`
`esibot_navigation2`	lancement Nav2, paramètres `esibot.yaml`, carte par défaut
`esibot_dashboard`	`imu_web_dashboard` (serveur HTTP de télémétrie)
`depth_cleanup`	bibliothèque C++/OpenCV de nettoyage métrique des images Kinect `16UC1`
`kinect_ros2` (externe)	pilote Kinect (v1), basé sur `libfreenect`
`m-explore-ros2` (externe)	exploration autonome par frontières au-dessus de Nav2 et `/map`
`esp32_camera_bridge` (externe)	pont MJPEG HTTP → ROS 2 `Image`

A.3 Sources

L'ensemble du contenu technique de ce rapport a été établi à partir des sources suivantes, fournies par l'équipe : le dépôt de code source `Esi_Bot` (branche `working_build`, paquets `esibot_*`), le pilote `kinect_ros2`, le firmware et le pont ROS 2 de l'ESP32-CAM (`esp32-camera`), ainsi que la fiche de spécification officielle du projet **EsiBot** (cahier des charges, comparatif TurtleBot3, liste des livrables) et les deux schémas d'architecture fournis par l'équipe (figures 2.1 et 2.2). La discussion Kinect versus capteur ultrasonique s'appuie aussi sur l'article de Kamarudin, Mamduh, Shakaff et Zakaria, *Performance Analysis of the Microsoft Kinect Sensor for 2D Simultaneous Localization and Mapping (SLAM) Techniques*, *Sensors* 14(12), 23365–23387, 2014, DOI `10.3390/s141223365`.